전북지역혁신플랫폼
Jeonbuk Regional Innovation Platform

JST

지능형모빌리티 설계

6주차 ROS2 프로그래밍 #4

1차시 TF2 (1)



* 본 영상은 무단 배포할 수 없습니다.

학습내용

1 TF2 실습 (1)

학습목표

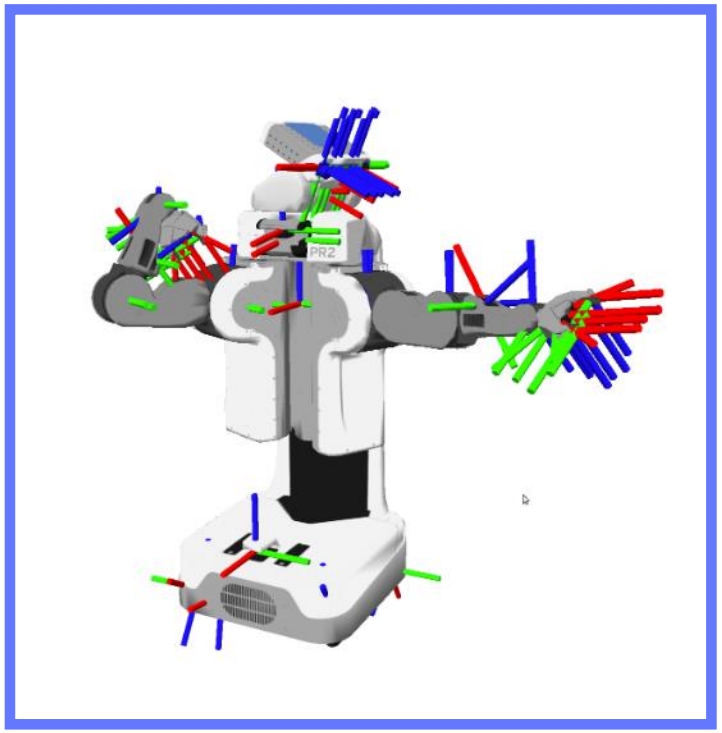
🎯 실습을 통해 TF2를 다룰 줄 안다.

TF2 실습 (1)

TF2란?

TF2 개요

- 로봇을 구성하는 Part의 Frame 간 변환을 다루는 라이브러리 로봇은 정말 많은 관절과 링크로 구성되어 있음
  - 로봇이 보고 있는 물체는 로봇의 Base로부터 어디쯤 위치할까?
  - Map Frame에 대해서 로봇의 Base는 어디쯤 위치할까?
- 실시간으로 변하는 로봇의 좌표계 사이의 Translation, Rotation 정보를 ROS Message를 통해 전달해줌

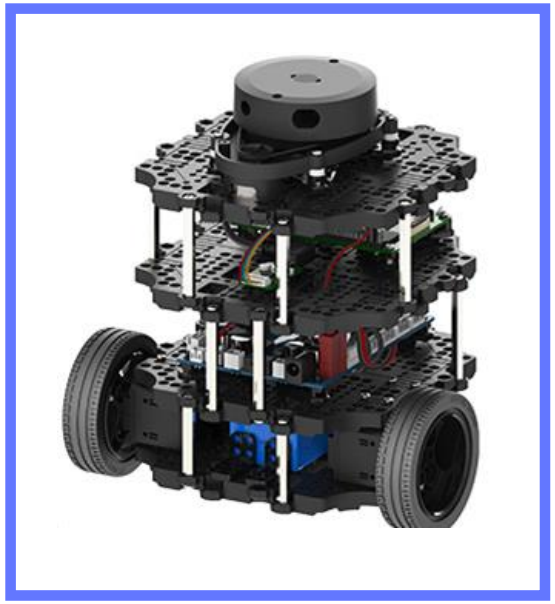


로봇을 구성하는 Frame

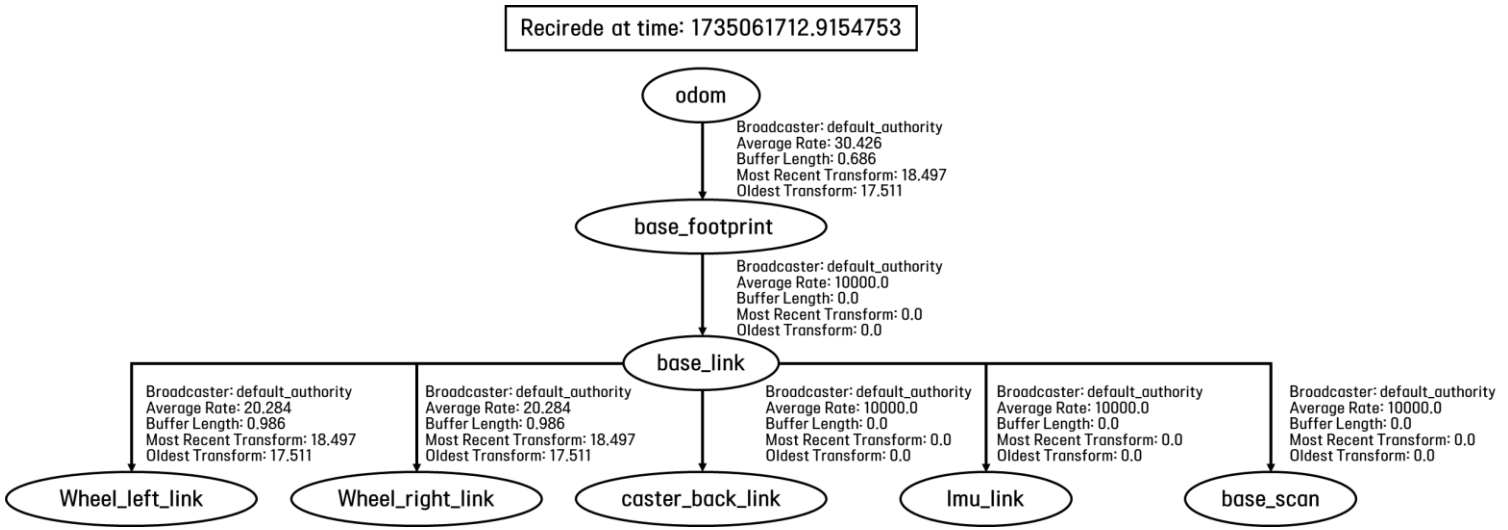
출처 https://accelerationrobotics.com/robotcore-transform.php

TF2란?

TF Tree



Turtlebot3



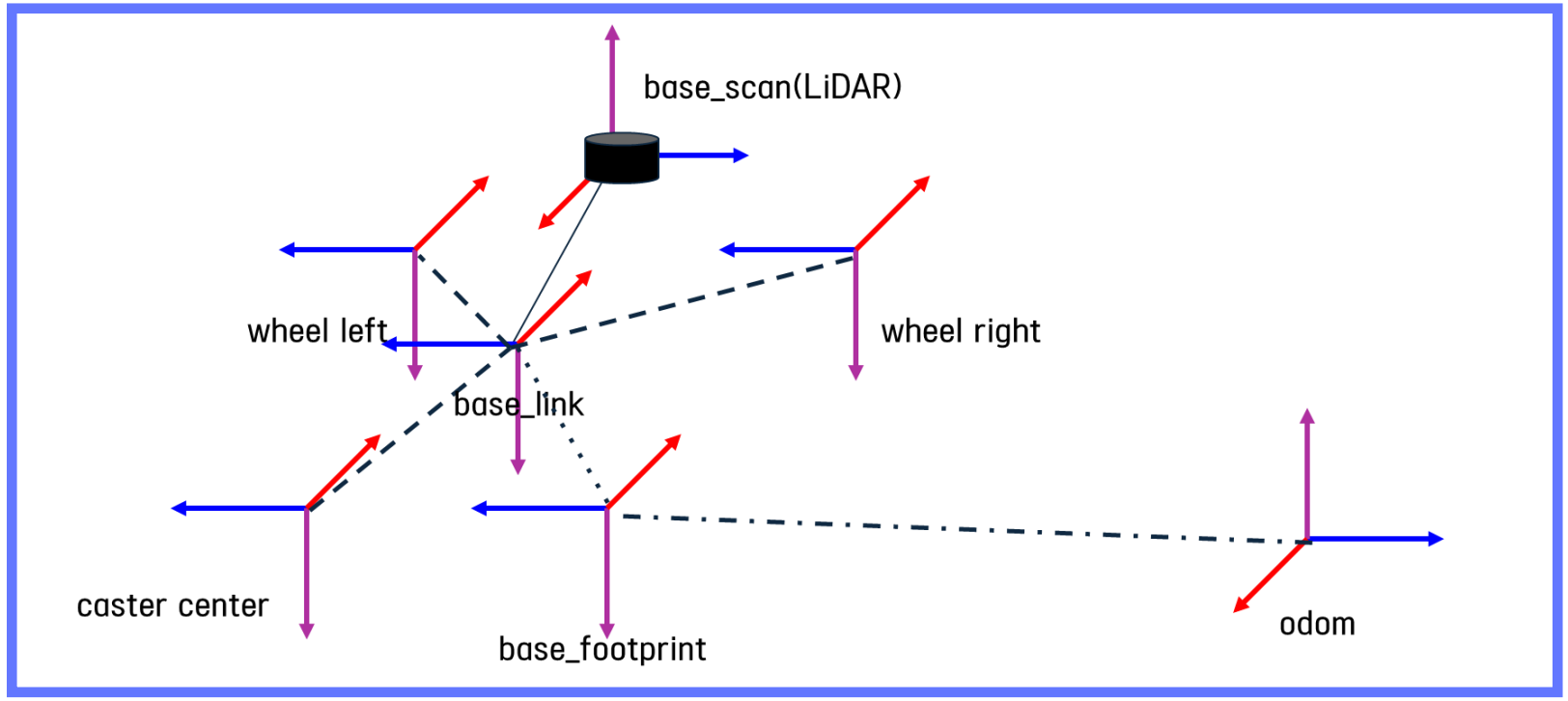
Turtlebot3 TF tree

출처 https://www.robotis.com/shop/item.php?it_id=901-0118-200

TF2란?

TF Tree

TF Tree를 그리면 아래와 같이 여러 프레임 간의 관계로 나타남



Transform Matrix

프레임 간 관계

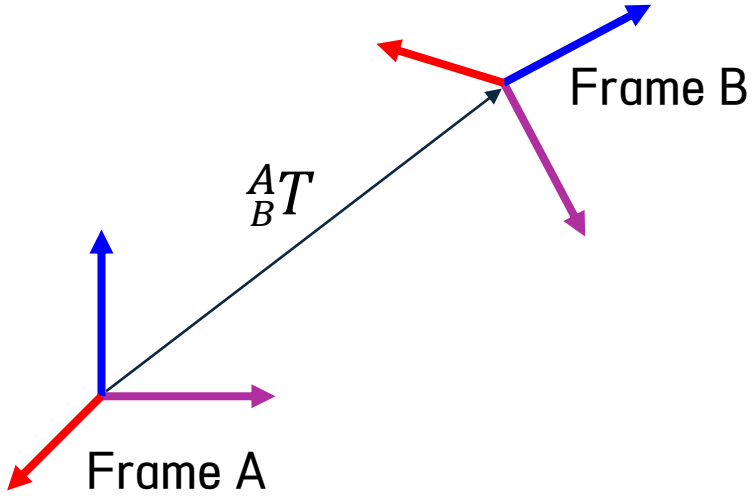
임의의 A Frame → B Frame 변환을 Transform Matrix로 표현 가능

- Frame A와 Frame B의 관계를 ${}^{A}_{B}T$라는 Homogeneous Matrix로 나타낼 수 있음

$$ {}^{A}_{B}T = \begin{bmatrix} {}^{A}_{B}R & {}^{A}_{B}P \\ 0 & 1 \end{bmatrix} $$

${}^{A}_{B}R$: Rotational Matrix
($R^{3\times3}$, 좌표계의 방향 관계)

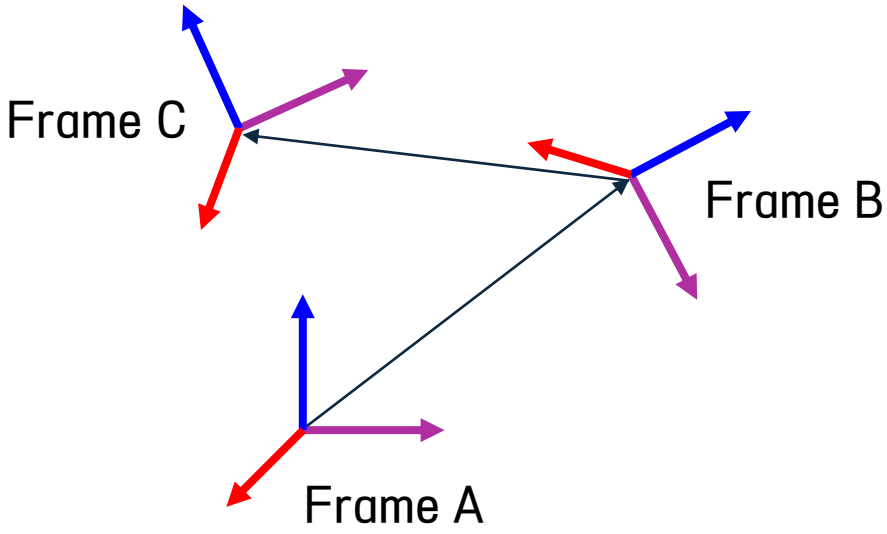
${}^{A}_{B}P$: Translation Vector
($R^{3\times1}$, 좌표계 간의 위치 관계)



Transform Matrix

프레임 간 관계

Frame C가 추가되면 세 프레임의 Homogeneous Matrix를 이용해 A to C의 관계를 알 수 있음



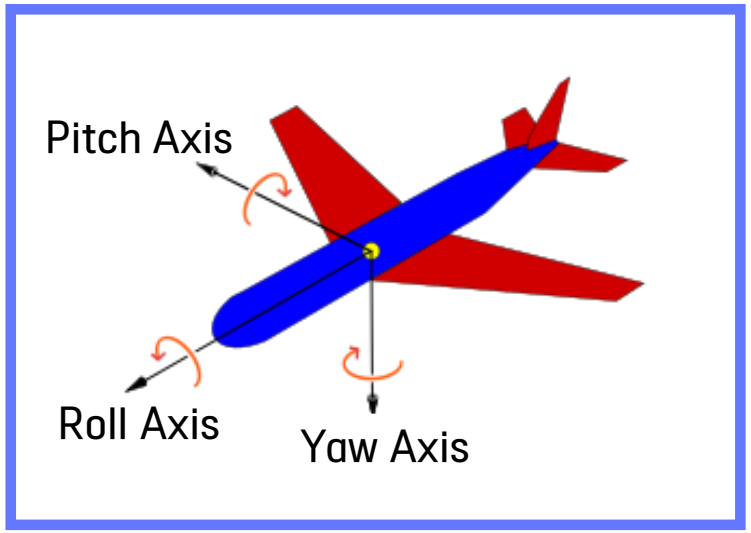
$$
{}^{A}_{C}T = {}^{A}_{B}T\,{}^{B}_{C}T
$$

$$
= \begin{bmatrix} {}^{A}_{B}R & {}^{A}_{B}P \\ 0 & 1 \end{bmatrix} \begin{bmatrix} {}^{B}_{C}R & {}^{B}_{C}P \\ 0 & 1 \end{bmatrix}
$$

Orientation Representation

오일러 각도(Euler Angles)

- Roll, Pitch, Yaw 세 개의 각으로 구성된 회전 표현법
- 짐벌락의 위험이 있으나 직관적으로 이해가 쉽다는 장점이 있음

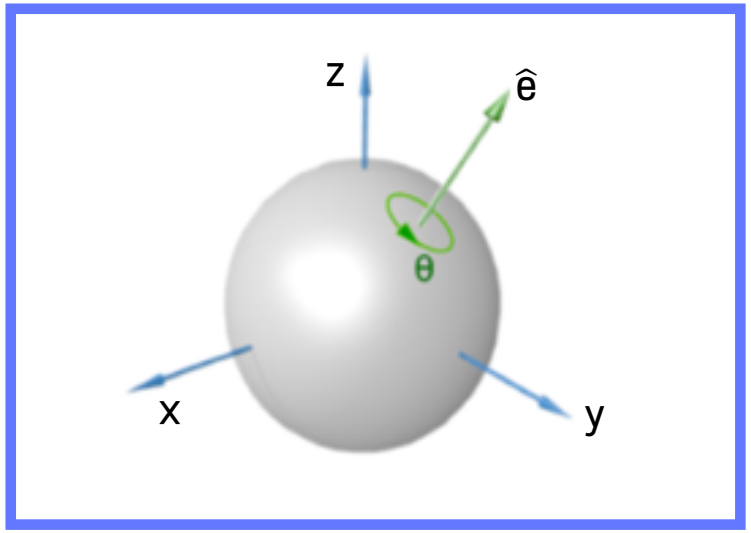


오일러 각

Orientation Representation

쿼터니언(Quaternion)

- x, y, z, w 네 개의 파라미터로 구성된 회전 표현법
- 짐벌락의 위험을 피하며 쿼터니언 곱으로 계산되기 때문에 효율적임
- 직관적으로 이해가 되지 않는다는 단점이 있음



쿼터니언

TF2 실습

좌표 변환 실습

```
cd ~/ros2_ws/src/ros2_urdf
mkdir launch
cd launch
gedit robot_launch.py
```

TF2 실습

좌표 변환 실습(robot_launch.py)

```
from ament_index_python.packages import get_package_share_path
from ament_index_python.packages import get_package_prefix

from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.conditions import IfCondition, UnlessCondition
from launch.substitutions import Command, LaunchConfiguration

from launch_ros.actions import Node
from launch_ros.parameter_descriptions import ParameterValue

import os
```

TF2 실습

좌표 변환 실습(robot_launch.py)

```
def generate_launch_description():
    package_name = "ros2_urdf"
    urdf_tutorial_path = get_package_share_path('urdf_tutorial')
    default_model_path = get_package_prefix(package_name)
    default_model_path = default_model_path.replace("install", "src")
    default_model_path = os.path.join(default_model_path,"robot/dof3_manipulator.urdf")

    default_rviz_config_path = urdf_tutorial_path / 'rviz/urdf.rviz'
    gui_arg = DeclareLaunchArgument(name='gui', default_value='true', choices=['true', 'false'],
                                    description='Flag to enable joint_state_publisher_gui')
    model_arg = DeclareLaunchArgument(name='model', default_value=str(default_model_path),
                                      description='Absolute path to robot urdf file')
    rviz_arg = DeclareLaunchArgument(name='rvizconfig', default_value=str(default_rviz_config_path),
                                     description='Absolute path to rviz config file')

    robot_description = ParameterValue(Command(['xacro ', LaunchConfiguration('model')]),
                                       value_type=str)
```

TF2 실습

좌표 변환 실습(robot_launch.py)

```
robot_state_publisher_node = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        parameters=[{'robot_description': robot_description}]
    )

    # Depending on gui parameter, either launch joint_state_publisher or joint_state_publisher_gui
    joint_state_publisher_node = Node(
        package='joint_state_publisher',
        executable='joint_state_publisher',
        condition=UnlessCondition(LaunchConfiguration('gui'))
    )

    joint_state_publisher_gui_node = Node(
        package='joint_state_publisher_gui',
        executable='joint_state_publisher_gui',
        condition=IfCondition(LaunchConfiguration('gui'))
    )
```

TF2 실습

좌표 변환 실습(robot_launch.py)

```
rviz_node = Node(
        package='rviz2',
        executable='rviz2',
        name='rviz2',
        output='screen',
        arguments=['-d', LaunchConfiguration('rvizconfig')],
    )

    return LaunchDescription([
        gui_arg,
        model_arg,
        rviz_arg,
        joint_state_publisher_node,
        joint_state_publisher_gui_node,
        robot_state_publisher_node,
        rviz_node
    ])
```

TF2 실습

좌표 변환 실습

터미널 입력

```
cd ~/ros2_ws/src/ros2_urdf
gedit package.xml
```

package.xml

```
...
  <exec_depend>ros2launch</exec_depend>
...
```

TF2 실습

좌표 변환 실습

터미널 입력

```
gedit CMakeLists.txt
```

CMakeLists.txt

```
...
install(DIRECTORY
launch
DESTINATION share/${PROJECT_NAME}
)
...
```

TF2 실습

좌표 변환 실습

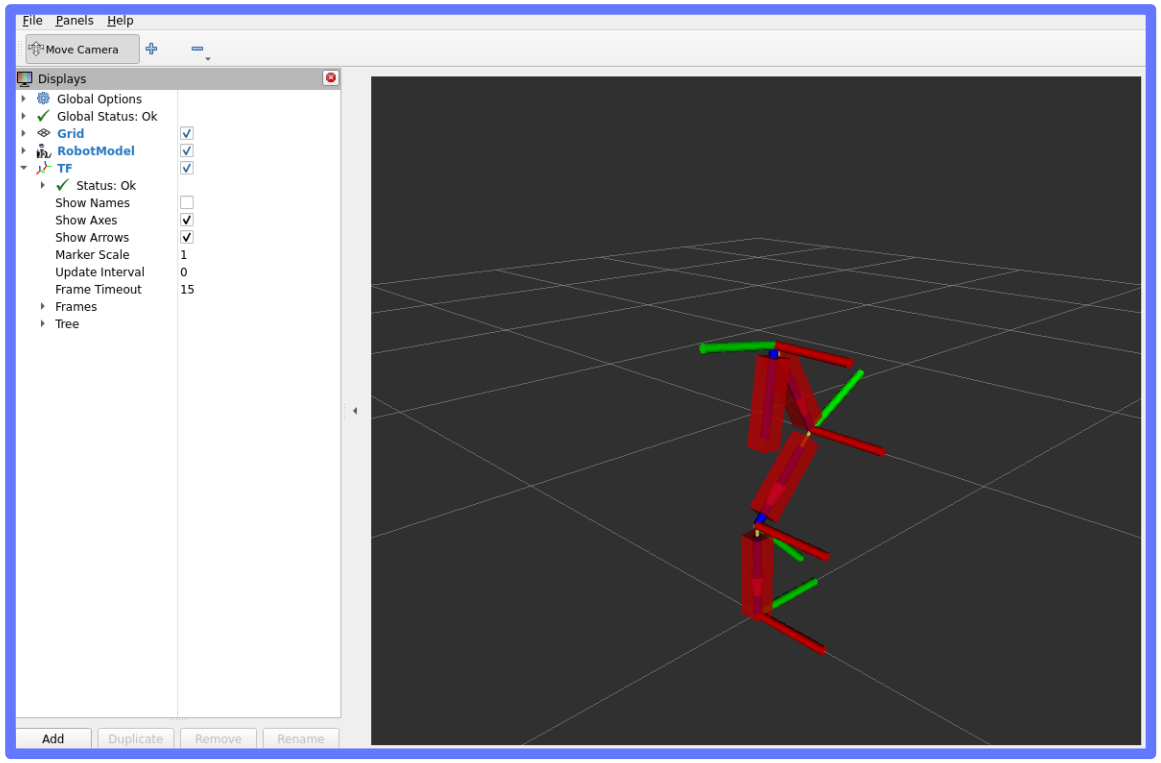
터미널 입력

```
sudo apt install ros-humble-urdf-*
cd ~/ros2_ws
colcon build --packages-select ros2_urdf
ros2 launch ros2_urdf robot_launch.py
```

TF2 실습

변환 확인하기

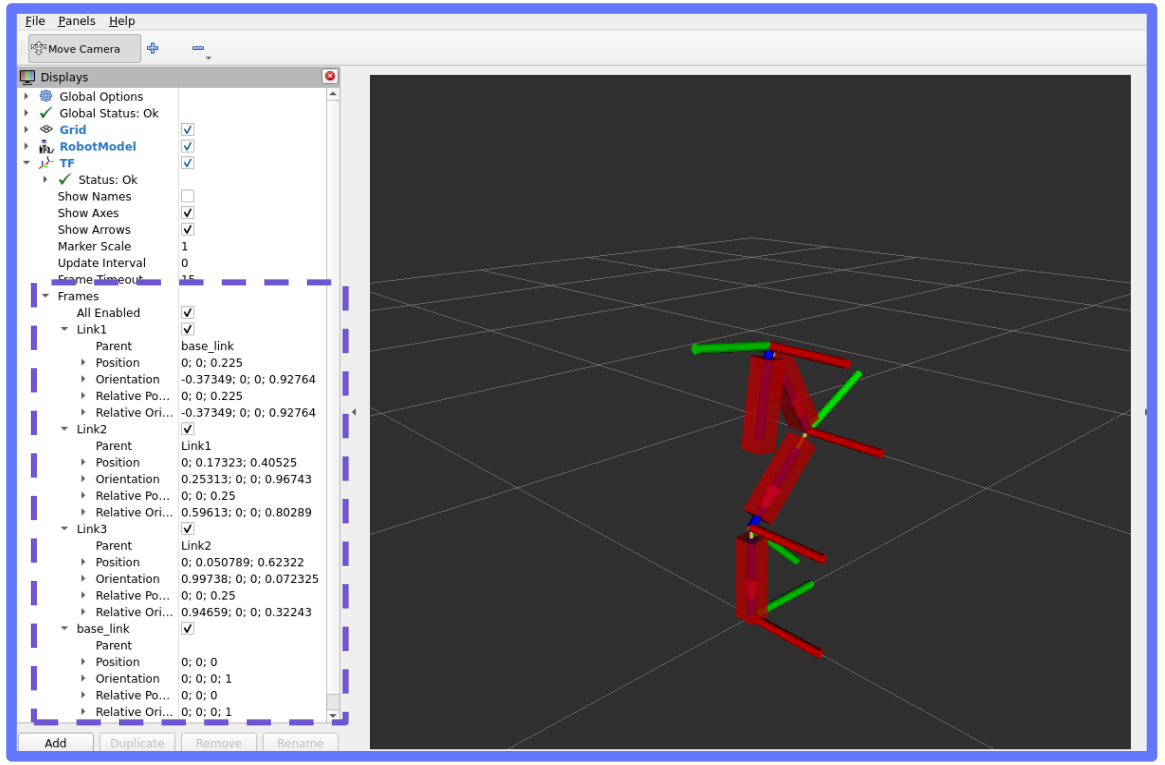
- rviz 화면의 좌측의 [Displays]의 TF 섹션 확인



TF2 실습

변환 확인하기

- [Frame]을 통해 현재 TF프레임의 상태를 확인 가능함
- 확인 가능한 상태
  - 링크 부모 관계
  - 절대위치
  - 절대방향
  - 상대위치
  - 상대방향



TF2 실습

변환 확인하기

- Joint State Publisher의 슬라이더를 조작해 각 Joint의 각도를 변경 가능
- 변경하면 [Display]의 [Frames]의 각 링크의 값이 변경되는 것을 볼 수 있음

